LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 7 – ABSTRACT & INTERFACE



NAMA: HUMAIRAH NURHAMIDAH

NIM: 103132400038

KELAS: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Kelas abstract adalah kelas yang dideklarasikan dengan kata kunci abstract dan berfungsi sebagai kerangka umum bagi kelas turunannya. Kelas ini tidak dapat diinstansiasi langsung, hanya turunan konkretnya yang bisa dijadikan objek. Di dalamnya boleh terdapat atribut, konstruktor, dan method biasa, serta abstract method yang hanya berisi deklarasi tanpa implementasi, misalnya earnings() pada Employee. Method abstrak ini wajib di-override oleh setiap subclass sesuai kebutuhannya masing-masing.

Interface adalah kontrak yang mendefinisikan method atau konstanta yang harus diimplementasikan oleh kelas lain melalui kata kunci implements. Atribut pada interface bersifat public static final, dan interface tidak memiliki konstruktor. Berbeda dengan kelas, interface mengizinkan *multiple inheritance*, sehingga satu kelas dapat mengimplementasikan beberapa interface sekaligus. Sejak Java 8, interface juga boleh memiliki default method, yaitu method berisi implementasi yang dipanggil lewat objek dan tidak wajib di-override, serta static method yang dipanggil lewat nama interface dan tidak dapat di-override. Keduanya memungkinkan penambahan fitur baru tanpa mengubah kelas yang sudah ada.

Perbedaan utamanya, abstract class boleh memiliki variabel dan method berimplementasi serta merupakan bagian dari hierarki pewarisan, sedangkan interface hanya berisi konstanta dan kontrak perilaku, tidak terikat hierarki tertentu, dan mendukung pewarisan jamak. Keduanya mendukung polimorfisme, yaitu satu referensi bertipe induk atau interface dapat menunjuk ke berbagai objek berbeda, dan Java secara otomatis menjalankan method yang sesuai dengan objek aslinya (*dynamic dispatch*).

B. Guided

1. Guided01

[Employee.java]

```
package Guided01;


public abstract class Employee {
    private String name;


    public Employee(String name){
        this.name = name;
```

```
    }


    public String getName(){
        return name;
    }


    //setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();


}
```

[SalariedEmployee.java]

```
package Guided01;

    //wajib menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

[Main.java]

```
package Guided01;

public class Main {

    public static void main(String[]args){
        SalariedEmployee salaried = new SalariedEmployee("Humairah", 5000000);
        System.out.println("Nama: "+salaried.getName());
```

```
        System.out.println("Gaji: Rp "+ salaried.earnings());
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul07 ---
Nama: Humairah
Gaji: Rp 5000000.0
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

2. Guided02

[Employee.java]

```
package Guided02;

public abstract class Employee implements Payable {

//fondasi umum yang belum memiliki peran spesifik sehingga tidak boleh
diinstansiasi langsung public abstract class Employee implements Payable{

    private String name;

    //constructur
    public Employee(String name) {
        this.name = name;
    }

    //getter
    public String getName() {
        return name;
    }

    //method, Setiap pegawai pasti punya gaji, tapi rumusnya berbeda-beda
    public abstract double earnings();

    //saat ada penagihan, alihkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount(){
        return earnings();
    }
}
```

[Invoice.java]

```
package Guided02;

   //Benda mati non-pegawai, tapii tetap diikat oleh aturan
public class Invoice implements Payable {
   private  String partName;
   private int quantity;
   private double pricePerItem;

   public Invoice(String partName, int quantity, double pricePerItem){
      this.partName = partName;
      this.quantity = quantity;
      this.pricePerItem = pricePerItem;
   }

   public String getPartName(){
      return partName;
   }

   @Override
   public double getPaymentAmount(){
      return quantity * pricePerItem;
   }
}
```

[Payable.java]

```
package Guided02;

//standar sistem: siapapu
public interface Payable {
   double getPaymentAmount();

}
```

[SalariedEmployee.java]

```
package Guided02;

//wajib menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {

   private double weeklySalary;

   public SalariedEmployee(String name, double weeklySalary) {
```

```
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

[Main.java]

```
package Guided02;

public class Main {
    public static void main(String[]args){
        Payable[] items = new Payable[2];
        items[0] = new Invoice("Monitor LED", 2, 1500000);
        items[1] = new SalariedEmployee("Humairah", 5000000);

    //Dynamic dispatch: Java otomatis mengeksekusi rumus yang tepat
    for (Payable item : items){
        System.out.println("Jumlah yang perlu dibayar: Rp" +
item.getPaymentAmount());
    }
}
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul07 ---
Jumlah yang perlu dibayar: Rp3000000.0
Jumlah yang perlu dibayar: Rp5000000.0
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

3. Guided03

[Employee.java]

```
package Guided03;

//fondasi umum yang belum memiliki peran spesifik sehingga tidak boleh
diinstansiasi langsung
```

```
public abstract class Employee implements Payable{
    private String name;

    //constructur
    public Employee(String name) {
        this.name = name;
    }

    //getter
    public String getName() {
        return name;
    }

    //method, Setiap pegawai pasti punya gaji, tapi rumusnya berbeda-beda
    public abstract double earnings();

    //saat ada penagihan, alihkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount(){
        return earnings();
    }
}
```

[Invoice.java]

```
package Guided03;

//Benda mati non-pegawai, tapii tetap diikat oleh aturan
public class Invoice implements Payable {
    private  String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
```

```
    }
}
```

[Payable.java]

```
package Guided03;

public interface Payable {
    double getPaymentAmount();

    // Fitur baru
    default void printReceipt(){
        System.out.println("Struk Resmi - Total: Rp" + getPaymentAmount());
    }

    // Fungsi utilitas
    static void logTransaction(String message){
        System.out.println("[AUDIT LOG] " + message);
    }
}
```

[SalariedEmployee]

```
package Guided03;

//wajib menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

[Main.java]

```
package Guided03;

public class Main {
```

```
    public static void main(String[] args) {
        Payable.logTransaction("Sistem pembayaran diinisialisasi.");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DDR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Andita", 5000000);

        for (Payable item : items){
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai diproses.");
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul07 ---
[AUDIT LOG] Sistem pembayaran diinisialisasi.
Struk Resmi - Total: Rp4000000.0
Struk Resmi - Total: Rp5000000.0
[AUDIT LOG] Transaksi selesai diproses.
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

Mahasiswa diminta membuat program Cinema POS (Point of Sale) menggunakan konsep Abstract Class dan Interface pada Java. Program ini digunakan untuk menghitung total pembayaran dari beberapa jenis transaksi, yaitu tiket bioskop reguler, tiket bioskop VIP, dan paket snack.

Program terdiri dari enam class, yaitu ItemTransaksi sebagai interface, TiketBioskop sebagai abstract class, TiketReguler, TiketVIP, PaketSnack, dan Main. Setiap jenis transaksi memiliki perhitungan nominal pembayaran masing-masing, tetapi seluruhnya menerapkan interface ItemTransaksi agar dapat ditampilkan dan dihitung melalui satu array yang sama.

Selain itu, program menerapkan method overriding, default method, dan static method untuk menampilkan rincian transaksi serta mencatat aktivitas kasir. Hasil akhirnya berupa rincian setiap transaksi dan total keseluruhan pembayaran.

Jawaban :

[ItemTransaksi.java]

```
package Unguided;


// Kontrak publik kasir: semua barang yang bisa dibayar wajib memenuhi ini
public interface ItemTransaksi {
    double getNominalBayar();
    String getNamaTransaksi();


    // Default method: cetak satu baris struk
    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi() + " | Total: Rp" +
getNominalBayar());
    }


    // Static method: fungsi utilitas log
    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

ModelRegresi

```
package Unguided;

public class ModelRegresi extends ModelML {
    // Turunan pertama: model regresi
    private int jumlahFitur;

    public ModelRegresi(String nama, int jumlahDataLatih, int jumlahFitur) {
        super(nama, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }
```

```
    @Override
    public void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // Overloading baru: dua parameter (epoch dan batch)
    public int hitungBiaya(int epoch, int batch) {
        return getJumlahDataLatih() * epoch / batch;
    }
}
```

[TiketBioskop.java]

```
package Unguided;

// Fondasi umum tiket, tidak boleh diinstansiasi langsung
public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return judulFilm;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }
```

```
    // Logika internal spesifik tiket, dirumuskan oleh subclass
    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();

    // Kontrak publik kasir, cukup meneruskan ke hitungTotalBayar()
    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

[TiketReguler.java]

```
package Unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi, int
nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
    }
}
```

[TiketVIP.java]

```
package Unguided;
public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP; // dikenakan per kursi

    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi, double
```

```
biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + biayaFasilitasVIP) * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + (getHargaDasar() +
biayaFasilitasVIP) + ")";
    }
}
```

[PaketSnack.java]

```
package Unguided;

// Bukan turunan TiketBioskop, tapi tetap bisa masuk kasir lewat ItemTransaksi
public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" +
hargaPerPorsi + ")";
    }
}
```

[Main.java]

```
package Unguided;

public class Main {
    public static void main(String[] args) {
        // Uji proteksi abstraksi: kalau di-uncomment, error karena TiketBioskop
abstract
        // TiketBioskop tiket = new TiketBioskop("Interstellar", 40000, 2);

        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan
#042.");

        TiketReguler reguler = new TiketReguler("Interstellar", 40000, 2, 3);
        TiketVIP vip = new TiketVIP("Dune: Part Two", 60000, 2, 25000);
        PaketSnack snack = new PaketSnack("Combo Popcorn Caramel + 2 Soda", 1,
55000);

        // Array heterogen bertipe interface
        ItemTransaksi[] keranjang = {reguler, vip, snack};

        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double grandTotal = 0;
        for (ItemTransaksi item : keranjang) {
            item.cetakRincianItem();
            grandTotal += item.getNominalBayar();
        }

        System.out.println("------------------------------------------------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + grandTotal);

System.out.println("============================================
================");

        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul07 ---
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Program ini terdiri dari enam berkas yang menghitung total belanja di kasir bioskop untuk dua jenis tiket dan satu paket snack. Interface ItemTransaksi menjadi kontrak kasir dengan dua method abstrak, yaitu getNominalBayar() dan getNamaTransaksi(). Interface ini juga memuat default method cetakRincianItem() untuk mencetak satu baris struk dan static method catatLogKasir() untuk mencetak log, sehingga kelas implementasi tidak perlu ditulis ulang. Kelas induk TiketBioskop bersifat abstract dan mengimplementasikan ItemTransaksi. Kelas ini menyimpan judul film, harga dasar, dan jumlah kursi sebagai atribut private yang diisi lewat konstruktor dengan this dan dibaca lewat getter. Method getNominalBayar() di kelas ini hanya meneruskan hasil hitungTotalBayar(), sedangkan hitungTotalBayar() dan getNamaTransaksi() dibiarkan abstract agar dirumuskan oleh kelas turunan.

TiketReguler mewarisi TiketBioskop dan menambah atribut nomor studio. Konstruktornya memanggil super(...) untuk mengisi atribut induk, lalu total dihitung dari harga dasar × jumlah kursi. TiketVIP bekerja dengan pola yang sama, dengan tambahan atribut biaya fasilitas VIP dan rumus total (harga dasar + biaya VIP) × jumlah kursi, karena biaya VIP dikenakan per kursi. Karena atribut induk bersifat private, kedua kelas turunan membacanya lewat getter. PaketSnack berdiri sendiri tanpa mewarisi TiketBioskop, tetapi tetap mengimplementasikan ItemTransaksi, dengan total jumlah porsi × harga per porsi.

Pada Main, instansiasi TiketBioskop ditulis sebagai komentar karena kelas abstract tidak bisa dibuat objeknya. Ketiga objek (Reguler, VIP, dan Snack) disimpan dalam satu array bertipe ItemTransaksi[] (polymorphism), sehingga di dalam perulangan setiap elemen otomatis menjalankan method versinya sendiri. Nominal tiap item dijumlahkan menjadi total 305000 (80000 + 170000 + 55000), dan log pembuka serta penutup dicetak lewat nama interface-nya.

D. Kesimpulan

Dari praktikum Modul 7 dapat disimpulkan bahwa abstract class dan interface sama-sama berfungsi untuk menerapkan abstraksi, tetapi dengan peran yang berbeda. Abstract class cocok dipakai sebagai kerangka bersama untuk kelas-kelas yang masih satu keluarga, karena dapat menyimpan atribut dan method berimplementasi, sekaligus memaksa kelas turunan untuk melengkapi method abstraknya. Interface cocok dipakai sebagai kontrak bagi kelas-kelas yang berasal dari hierarki berbeda, dan sejak Java 8 dapat diperkaya dengan default method dan static method tanpa mengubah kelas yang sudah mengimplementasikannya.

Pada soal unguided, konsep tersebut diterapkan dalam sistem kasir bioskop. TiketBioskop sebagai abstract class menjadi induk bagi TiketReguler dan TiketVIP, sedangkan interface ItemTransaksi menyatukan tiket dan PaketSnack yang tidak sekeluarga. Dengan array bertipe ItemTransaksi[], seluruh item dapat diproses dalam satu perulangan, dan setiap objek otomatis menjalankan versi method-nya sendiri berkat polimorfisme. Hasilnya, program menghasilkan total transaksi Rp305000 tanpa kasir perlu membedakan jenis item satu per satu.